# Agent IA de revue des Pull Requests

État d'avancement et suite du projet · 11 novembre 2026

L'agent relit automatiquement chaque Pull Request GitHub : il filtre le diff, détecte les secrets, fait relire le code par un modèle d'IA hébergé par Calipso, calcule un verdict par des règles fixes, publie le résultat sur la PR et alerte par email en cas de PR bloquante. Il ne fusionne jamais rien et ne modifie aucun code ; toute analyse est journalisée.

## Synthèse

| 8 / 8 | 12 / 12 | ≈ 77 % | 0 | 2 min 22 |
|---|---|---|---|---|
| défauts plantés détectés (sur deux séries) | verdicts conformes (sur deux séries) | remarques pertinentes (objectif : 70 %) | secret retrouvé dans le journal | durée p95 par PR (3 workers, attente comprise) |

- **Fonctionne de bout en bout sur le vrai GitHub** : GitHub App dédiée, 3 dépôts de test, 12 PR ouvertes en même temps, deux fois.
- **Version 2 terminée** : verdict à 4 états, mode pilote (l'IA ne bloque pas seule), interrupteur de publication, journal des analyses et interface de suivi en français.
- **Un défaut réel trouvé et corrigé** grâce aux tests en conditions réelles (fenêtre du modèle Qwen3).
- **Pas encore en production** : le dispositif de test (relais smee.io, modèle en HTTP) ne convient qu'à du code fictif. Le déploiement demande HTTPS, un réseau privé et la validation de Hicham.

## 1. Ce qui est réalisé

### 1.1 Prototype (semaines 1 et 2)

- Chaîne complète : webhook GitHub → diff filtré → détection de secrets → revue IA → verdict → commentaires et check sur la PR → email d'alerte → journal d'audit.
- Droits minimaux : le bot publie des revues « commentaire » uniquement, jamais d'approbation ni de fusion.
- Aucun code ne sort de l'infrastructure : liste blanche des hôtes autorisés pour le modèle.

### 1.2 Choix du modèle

Le modèle Qwen3-14B de Calipso a été testé sur les points d'accès */chat* et */think*, et comparé à d'autres modèles (Qwen3-Coder-Next, Qwen3-Coder-30B-A3B) sur du code fictif. **Retenu : Qwen3-14B /chat**, meilleur rapport qualité / matériel, 20 à 40 s par PR, sans erreur serveur.

### 1.3 Version 2

| Domaine | Ce qui a été fait |
|---|---|
| Verdict | 4 états calculés par des règles fixes : **Bloquante**, **Analyse incomplète**, **À vérifier**, **Contrôles satisfaits**. Version des règles affichée dans chaque résultat. |
| Mode pilote | Les remarques de l'IA sont des avis : seule une détection déterministe (secret) peut rendre une PR bloquante. |
| Sécurité | Secrets recherchés aussi dans le titre et la description de la PR ; contenu masqué avant tout stockage ; commit re-vérifié avant publication. |
| Pilotage | Interrupteur de publication : coupe toute écriture GitHub et tout email, sans redéploiement, visible dans l'interface. |
| Performance | Lots du diff analysés en parallèle (limite de concurrence), temps d'attente en file mesuré. |

| Domaine | Ce qui a été fait |
|---|---|
| Suivi | Journal d'événements rejouable ; interface en français protégée par connexion : vue en direct, page d'analyse, historique filtrable, statistiques, avis « pertinente / fausse alerte ». |

### 1.4 Raccordement au vrai GitHub

- GitHub App **calipso-revue-pr-test** : checks et PR en écriture, contenu en lecture, rien d'autre.
- Organisation de test **test-agent-revue** avec 3 dépôts privés.
- Application Symfony de test « Portefeuille démo » (contrats fictifs, 2 rôles, 19 tests, CI GitHub Actions).
- Webhooks relayés vers le poste par smee.io (signature vérifiée par l'agent ; le code n'y transite pas).

### 1.5 Banc de test sur le vrai GitHub

12 cas ont été écrits avec leur résultat attendu : 8 défauts (injection SQL, contrôle de droits manquant, *dump()* oublié, clé d'API en dur, montant en flottant, référence en doublon, faille XSS, mot de passe dans la description), 3 PR propres et 1 fichier trop volumineux. Les 12 PR ont été ouvertes en moins de 5 secondes sur les 3 dépôts, une fois avec 1 worker, une fois avec 3 workers.

| Indicateur | Série 1 · 1 worker | Série 2 · 3 workers |
|---|---|---|
| Verdicts conformes | **12 / 12** | **12 / 12** |
| Défauts attendus détectés | **8 / 8** | **8 / 8** |
| PR propres sans remarque majeure | 2 / 3 | **3 / 3** |
| Check GitHub conforme au verdict | 12 / 12 | 12 / 12 |
| Email d'alerte si et seulement si bloquante | 12 / 12 | 12 / 12 |
| Secrets retrouvés dans le journal | 0 | 0 |
| Erreurs du modèle | 0 / 11 lots | 0 / 11 lots |
| Durée totale p50 / p95 (attente comprise) | 3 min 55 / 5 min 39 | **1 min 08 / 2 min 22** |
| Exécution seule p50 / p95 | 24 s / 1 min 04 | 32 s / 1 min 00 |

- **Pertinence ≈ 77 %** : 20 remarques utiles sur 26 (dont un vrai défaut non prévu : formulaire de suppression sans jeton CSRF). 3 fausses alertes et 2 remarques sans intérêt.
- **Mode pilote respecté** : les défauts trouvés par l'IA donnent « À vérifier » ; seuls les deux secrets ont bloqué la PR et déclenché l'email.
- **Charge** : l'attente en file domine. 3 workers divisent la durée p95 par 2,4 ; le serveur Qwen3 a tenu 9 requêtes simultanées sans ralentir nettement.

## Cas limites vérifiés

| Situation | Comportement observé |
|---|---|
| Nouveau commit poussé pendant l'analyse | Check de l'ancien commit clos « remplacé », rien publié pour lui ; le nouveau commit a sa propre revue. |
| PR fermée puis rouverte (même commit) | Pas de seconde analyse. |
| Webhook falsifié ou sans signature | Refusé (401). |
| Interrupteur de publication coupé | Analyse journalisée, aucune écriture GitHub ; réactivé : publication normale. |
| Fichier trop volumineux pour GitHub | « Analyse incomplète », jamais vert. |

### 1.6 Défaut trouvé et corrigé

**Première PR réelle : erreur 502 du modèle.** La fenêtre du modèle Qwen3 servi est de 8 192 jetons (question + réponse) et l'agent demandait toujours 6 144 jetons de réponse. Correction : la taille de la réponse est maintenant calculée à chaque appel, avec une nouvelle tentative réduite si le serveur refuse. Avant la correction, l'agent avait bien réagi : « Analyse incomplète », jamais de faux vert. 77 tests passent.

## 2. Points en attente de décision

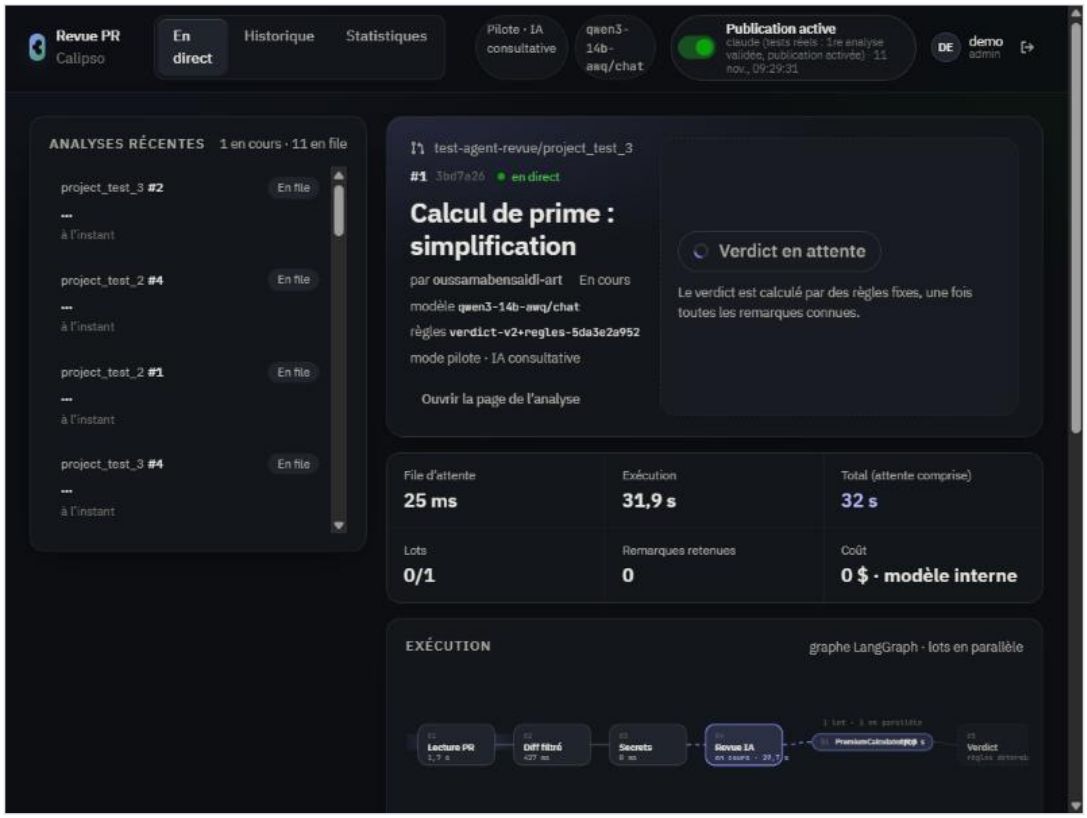
| Sujet | Décision attendue |
|---|---|
| Enregistrement du code | Valider l'enregistrement (commit) de la version 2, du correctif et du banc : rien n'est encore enregistré dans le dépôt de l'agent. |
| Captures GitHub | Se connecter à GitHub dans le navigateur de test pour capturer les pages de PR (dépôts privés). |
| « Tester la capture d'écran » | Préciser le besoin : captures comme preuves (fait pour l'interface) ou nouvelle fonction où l'agent fait des captures de l'application relue. |

## 3. Ce qui sera fait ensuite

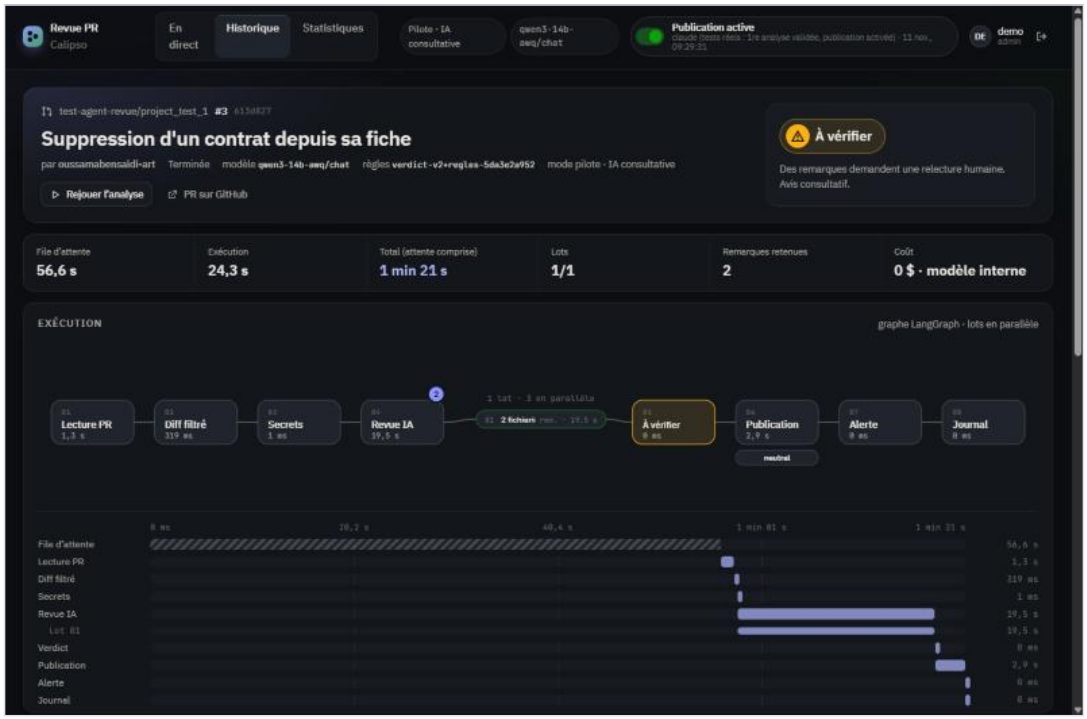
| Étape | Contenu | Condition |
|---|---|---|
| **1. Contrôles statiques** (semaines 3-4, priorité Hicham) | PHPStan, PHP-CS-Fixer, PHPUnit, *composer audit* et gitleaks exécutés dans un conteneur isolé par PR. Résultats fiables, donc capables de bloquer une PR, contrairement à l'IA. | — |
| **2. Benchmark 30 à 50 PR** | Comparer Qwen3-14B, Qwen3-Coder-30B-A3B et Qwen3-Coder-Next sur des PR réelles de l'équipe, pas seulement sur nos cas écrits. | GPU suffisant pour Coder-Next |
| **3. Moins de fausses alertes** | Donner du contexte à l'IA (fichiers voisins, conventions Symfony), relecteurs spécialisés (sécurité, tests), ajustement des consignes à partir des avis saisis dans l'interface. | — |
| **4. Fiabilisation** | File d'attente avec acquittement et nouvelles tentatives, traitement des analyses en échec ; revue mise à jour à chaque push au lieu d'en ajouter une ; fermer les deux limites connues (check resté « en cours » si l'interrupteur est coupé en pleine analyse, court délai entre re-vérification et publication). | — |
| **5. Déploiement** | Serveur Calipso en HTTPS, webhook direct vers l'agent (sans relais), modèle sur réseau privé, gestion des secrets, cookies sécurisés, supervision et sauvegardes. Préparé par nous, déployé par un humain. | Validation Hicham |
| **6. Pilote supervisé** | Quelques dépôts réels de l'équipe en mode pilote ; suivi de la pertinence par les avis de l'équipe, puis décision collective sur l'autorisation de blocage par l'IA. | Après déploiement |

**Limites du banc actuel.** 12 cas distincts joués deux fois, écrits par l'équipe qui connaît l'agent (défauts plutôt nets) ; une seule application cible ; un seul modèle. Le relais smee.io et l'API du modèle en HTTP sur IP publique ne conviennent qu'à du code fictif, jamais à du code Calipso.

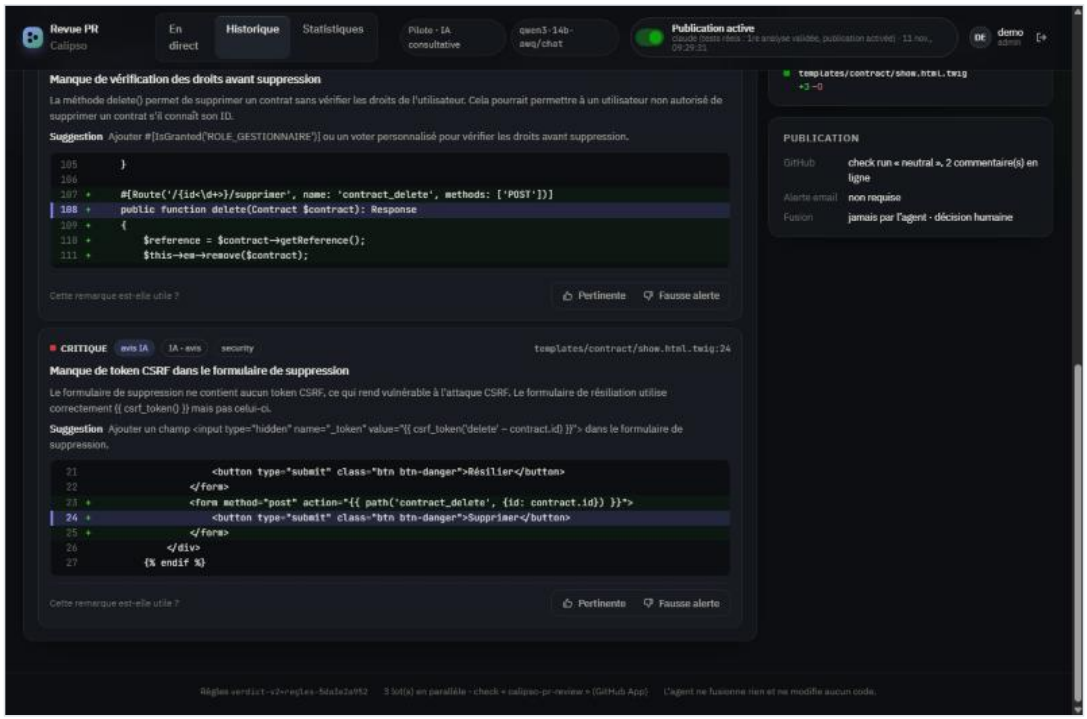
# Annexe · captures de l'interface de suivi



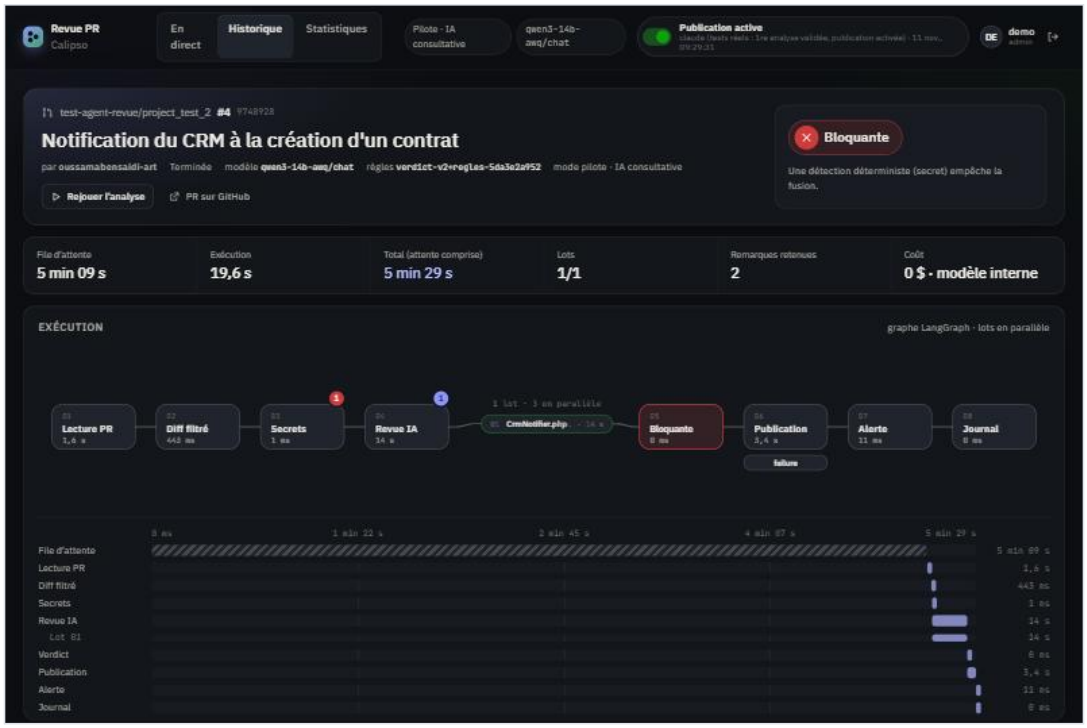

*Vue en direct pendant la série 1 : une analyse en cours, onze en file d'attente.*



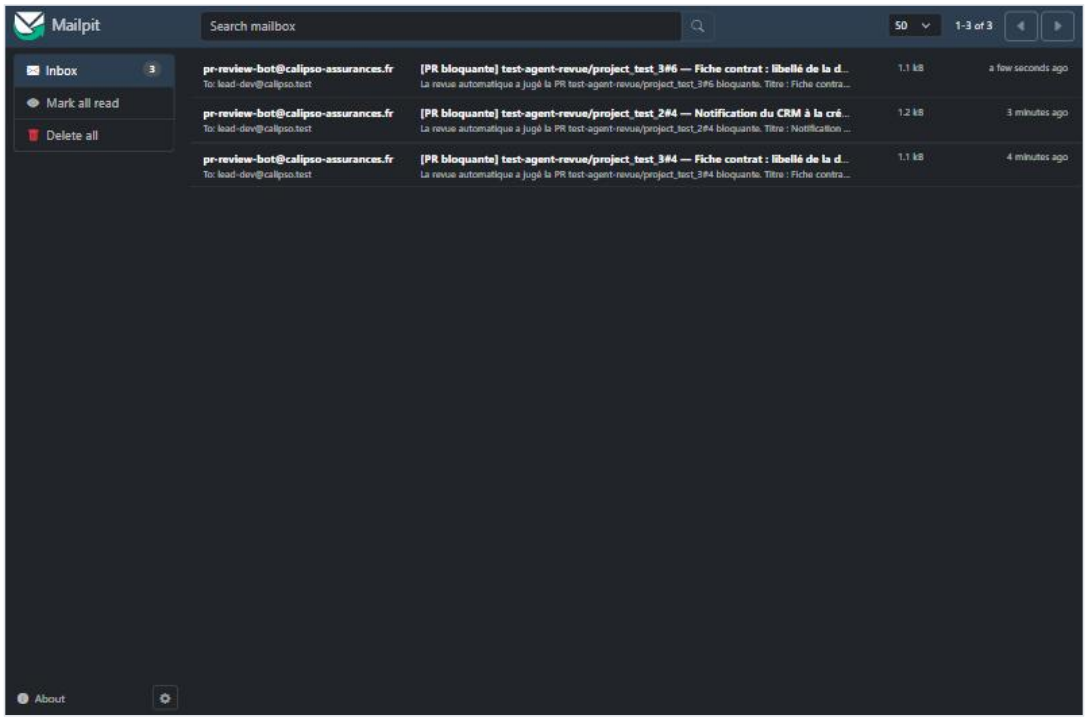

*Analyse d'une PR « suppression sans contrôle de droits » : graphe d'exécution, attente 57 s, exécution 24 s, verdict « À vérifier ».*




*Remarques de l'IA sur la même PR, avec l'extrait du diff et les boutons d'avis « Pertinente / Fausse alerte ».*




*PR contenant une clé d'API en dur : verdict « Bloquante » par la détection de secrets, check GitHub en échec et alerte envoyée.*




*Emails d'alerte reçus pour les PR bloquantes (boîte de test locale, aucune vraie adresse).*